# Ingegneria del Software – 08/07/2025 – Soluzione

| **Consegnare SOLO i fogli del tema.** | **Attaccare l'etichetta in questa zona.** |
|---|---|
| Non separarli. Altri fogli saranno ignorati.<br>I fogli sono fronte/retro. 2 ore di tempo.<br>Si possono consultare libri e appunti.<br>Tutti i dispositivi elettronici sono vietati. | Compiti senza etichetta non sono validi. |

# Esercizio 1

Si consideri la classe Java `ActivityManager` per gestire le attività sportive di un utente.

```
public class ActivityManager {
  // Restituisce l'insieme delle attivita' dell'utente, organizzate per tipologia.
  public /*@ pure @*/ Map<ActivityType, Set<Activity>> getActivities() { ... }

  // Aggiunge l'activity passata come parametro.
  public void addActivity(Activity activity) { ... }

  // Restituisce le attivita' svolte nel periodo tra le date start e end (entrambe incluse).
  public /*@ pure @*/ List<Activity> getActivitiesBetween(Date start, Date end) { ... }

  // Restituisce le tipologie di attivita' svolte nel periodo tra le date start e end (entrambe incluse).
  // Le tipologie sono ordinate per numero di attivita' svolte nel periodo indicato (dalla tipologia
  // meno praticata alla tipologia piu' praticata).
  public /*@ pure @*/ List<ActivityType> getActivityTypesBetween(Date start, Date end) { ... }
}

public class Activity {
  // Ritorna la tipologia di attivita'
  public /*@ pure @*/ ActivityType getType() { ... }

  // Ritorna la data in cui e' stata svolta l'attivita'
  public /*@ pure @*/ Date getDate() { ... }
}

public class Date implements Comparable<Date> { ... }
```

**Domanda a)** Si specifichi in JML il metodo `getActivitiesBetween()`.

**Soluzione**

```
//@ requires start != null && end != null && start.compareTo(end) <= 0
//@ ensures \result != null &&
//@ (\forall Activity a; ; \result.contains(a) <==>
//@   (\exists ActivityType t; getActivities().containsKey(t) ; getActivities().get(t).contains(a) &&
//@     a.getDate().compareTo(start) >= 0 && a.getDate().compareTo(end) <= 0) ) &&
//@ (\forall int i; i >= 0 && i < \result.size();
//@   (\forall int j; j >= 0 && j < i; !\result.get(i).equals(\result.get(j))) )
public /*@ pure @*/ List<Activity> getActivitiesBetween(Date start, Date end);
```

**Domanda b)** Si specifichi in JML il metodo `getActivityTypesBetween()`.

**Soluzione**

```
//@ requires start != null && end != null && start.compareTo(end) <= 0
//@ ensures \result != null && (\forall ActivityType t; ; \result.contains(t) <==>
//@   (\exists Activity a; getActivitiesBetween(start, end).contains(a); a.getType().equals(t)) ) &&
//@ (\forall int i; i >= 0 && i < \result.size() - 1;
//@   (\num Activity a; getActivitiesBetween(start, end).contains(a); a.getType().equals(\result.get(i))) <=
//@   (\num Activity a; getActivitiesBetween(start, end).contains(a); a.getType().equals(\result.get(i+1))) ) &&
//@ (\forall int i; i >= 0 && i < \result.size();
//@   (\forall int j; j >= 0 && j < i; !\result.get(i).equals(\result.get(j))) )
public /*@ pure @*/ List<ActivityType> getActivityTypesBetween(Date start, Date end);
```

**Domanda c)** Si specifichi in JML il metodo `addActivity`.

**Soluzione**

```
//@ requires activity != null;
//@ ensures (\forall ActivityType t; ;
//@   !t.equals(activity.getType()) ==> (
//@     getActivities().containsKey(t) <==> \old(getActivities().containsKey(t)) &&
//@     getActivities().containsKey(t) ==> (
//@       getActivities().get(t).size() == \old(getActivities().get(t)).size() &&
//@       getActivities().get(t).containsAll(\old(getActivities().get(t))) ) ) &&
```

```
//@   t.equals(activity.getType()) ==> (getActivities()().containsKey(t) &&
//@     getActivities().get(t).size() == \old(getActivities().get(t)).size() + 1 &&
//@     getActivities().get(t).containsAll(\old(getActivities().get(t))) &&
//@     getActivities().get(t).contains(activity) ) )
public void addActivity(Activity activity);
```

**Domanda d)** Si consideri una classe `ActivityManager2` che aggiunge un metodo `void addIfFirst(Activity a)` che aggiunge l'attività `a` solo se questa è la prima della sua tipologia. Può `ActivityManager2` essere considerato una sottoclasse di `ActivityManager` in accordo con il principio di sostituzione di Liskov?

**Soluzione**

Si, in quanto si tratta di un'estensione pura (aggiunta di un metodo), che non modifica le proprietà invarianti o evolutive della classe. Il metodo `addIfFirst()` potrebbe essere implementato a partire dai metodi di `ActivityManager`, controllando prima che l'attività sia la prima del suo tipo (mediante il metodo `getActivities()`) e inserendo (mediante il metodo `addActivity()`) solo se il controllo ha dato esito positivo.

# Esercizio 2

Si consideri la seguente classe `ActivityManagerSync`, versione semplificata e sincronizzata della precedente.

```
class ActivityManagerSync {
  private Map<ActivityType, List<Activity>> activities;
  public ActivityManagerSync() {
    activities = new HashMap<ActivityType, List<Activity>>();
  }
  public void addActivity(Activity act) {
    List<Activity> s = null;
    synchronized(activities) {
      s = activities.get(act.getType());
      if(s == null) {
        s = new ArrayList<Activity>();
        activities.put(act.getType(), s);
      }
    }
    synchronized(s) {  s.add(act);  }
  }
}
```

**Domanda a)** La classe è correttamente sincronizzata? In particolare, si evita l'accesso concorrente a zone di memoria condivise? Motivare la propria risposta e in caso di risposta negativa spiegare cosa andrebbe cambiato per evitare conflitti.

**Soluzione**

La classe è correttamente sincronizzata. L'accesso alla mappa è protetto acquisendo il lock sulla mappa stessa, mentre l'accesso ai valori della mappa (le liste) è protetto acquisendo il lock sul valore medesimo, prima di aggiungervi l'attività passata come parametro.

**Domanda b)**

Si consideri una nuova versione della classe `ActivityManagerSync` che aggiunge l'istruzione `activities.notifyAll()` al metodo `addActivity` e introduce un nuovo metodo `getDateLastActivity` che ritorna la data dell'ultima attività di tipo `type`, sospendendo il chiamante se non esiste la chiave `type` cercata. Il nuovo metodo ha un problema di sincronizzazione. Si illustri che tipo di errore può emergere e in quali condizioni (non serve correggere il problema).

```
class ActivityManagerSync {
  private Map<ActivityType, List<Activity>> activities;
  public ActivityManagerSync() {
    activities = new HashMap<ActivityType, List<Activity>>();
  }
  public void addActivity(ActivityType type, Activity act) {
    List<Activity> s = null;
    synchronized(activities) {
      s = activities.get(type);
      if(s == null) {
        s = new ArrayList<Activity>();
        activities.put(type, s);
        activities.notifyAll()
      }
    }
    synchronized(s) { s.add(act); }
  }
  public Date getDateLastActivity(ActivityType type) throws InterruptedException {
    List<Activity> s = null;
    synchronized(activities) {
      while( ! activities.containsKey(type) ) activities.wait();
      s = activities.get(type);
    }
    synchronized(s) {  return s.get(s.size()-1).getDate();  }
  }
}
```

**Soluzione**

Nel caso in cui un thread T1 stia aggiungendo (metodo `addActivity`) la prima attività di un nuovo tipo, mentre un thread T2 invoca il metodo `getDateLastActivity` per lo stesso tipo, potrebbe succedere che T1 venga interrotto da T2 alla fine del blocco sincronizzato su `activities`, prima di ottenere il lock sulla nuova lista (al momento vuota). A questo punto T2 potrebbe eseguire tutto il metodo `getDateLastActivity`, dando luogo ad una `IndexOutOfBoundException` in corrispondenza dell'ultima istruzione (la lista associata al nuovo tipo è, come detto, presente ma ancora vuota perché T1 non ha ancora eseguito `s.add(act)`).

**Domanda c)**

Si aggiunga alla classe `ActivityManagerSync` originale un nuovo metodo `printNumAct(ActivityType type)` che crea e avvia un thread, separato rispetto al chiamante, al fine di stampare il numero di attività registrate per il tipo dato.

**Soluzione**

```
public void printNumAct(ActivityType type) {
  new Thread( () -> {
    List<Activity> s = null;
    synchronized(activities) {
      s = activities.get(type);
    }
    if(s==null) System.out.println(0);
    else {
      synchronized(s) {
        System.out.println(s.size());
      }
    }
  } ).start();
}
```

# Esercizio 3

Si consideri la seguente versione semplificata (avente il solo metodo `getActivities`) della classe `ActivityManager` introdotta nel primo esercizio.

```
public class ActivityManager {
  public Map<ActivityType, Set<Activity>> getActivities() { ... }
}

public class Activity {
  public ActivityType getType() { ... }
  public Date getDate() { ... }
}

public class Date implements Comparable<Date> { ... }
```

Si implementino i seguenti metodi statici *facendo uso esclusivamente dei costrutti della programmazione funzionale in Java*.

**Domanda a)** Si completi il metodo `getActivitiesBetween()` che ritorna l'insieme di tutte le attività all'interno di `m` che si trovano tra le date `start` e `end` (estremi inclusi).

```
public static Set<Activity> getActivitiesBetween(ActivityManager m, Date start, Date end) { ... }
```

**Soluzione**

```
public static Set<Activity> getActivitiesBetween(ActivityManager m, Date start, Date end) {
  return m.getActivities().values().stream()
    .flatMap(s -> s.stream())
    .filter(a -> a.getDate().compareTo(start) >= 0)
    .filter(a -> a.getDate().compareTo(end) <= 0)
    .collect(Collectors.toSet());
  }
```

**Domanda b)** Si completi il metodo `popularTypes()` che ritorna la lista di tutte le tipologie di attività presenti all'interno di `m`, ordinate in base al numero di attività svolte per quella tipologia (dalla tipologia con meno attività alla tipologia con più attività).

```
public static List<ActivityType> popularTypes(ActivityManager m) { ... }
```

**Soluzione**

```
public static List<ActivityType> popularTypes(ActivityManager m) {
  return m.getActivities().keySet().stream()
    .sorted((t1, t2) -> Integer.compare(
      m.getActivities().get(t1).size(),
      m.getActivities().get(t2).size()))
    .toList();
}
```

# Esercizio 4

Si considerino le seguenti classi Java:

```
abstract class Activity {
  protected double duration; // in minutes
  public Activity(double duration) {
    this.duration = duration;
  }
  public void merge(Activity other) {
    this.duration = this.duration + other.duration;
  }
  public abstract void mergeAndPrint(Activity other) ;
}

class Running extends Activity {
  public Running(double duration) {  super(duration);  }
  public void mergeAndPrint(Activity other) {
    this.merge(other);
    System.out.println("Tot duration "+this.duration);
  }
}

class OutdoorRunning extends Running {
  public OutdoorRunning(double duration) {  super(duration);  }
  public void merge(Running other) {
    this.duration = this.duration + other.duration + 5.0;
  }
}

class SkyRunning extends OutdoorRunning {
  public SkyRunning(double duration) {  super(duration);  }
  public void mergeAndPrint(OutdoorRunning other) {
    this.merge(other);
    System.out.println("Tot duration "+this.duration);
  }
}

class Swimming extends Activity {
  public Swimming(double duration) {  super(duration);  }
  public void mergeAndPrint(Activity other) {
    this.merge(other);
    System.out.println("Tot duration "+(this.duration+10)+" (adding 10 extra for transition)");
  }
}
```

e il seguente frammento di codice le cui righe sono state numerate per riferimento:

```
1  Activity a1 = new Running(60.0);
2  Activity a2 = new Swimming(60.0);
3  Swimming s1 = new Swimming(60.0);
4  Swimming s2 = (Swimming) new Running(60.0);
5  Running  r1 = new OutdoorRunning(60.0);
6  Running  r2 = new SkyRunning(60.0);
7  OutdoorRunning or1 = new OutdoorRunning(60.0);
8  OutdoorRunning or2 = new OutdoorRunning(60.0);
9  SkyRunning sr1 = new SkyRunning(60.0);
10 SkyRunning sr2 = new SkyRunning(60.0);
11
12 a1.mergeAndPrint(a2);
13 a2.mergeAndPrint(s1);
14 s1.mergeAndPrint(s2);
15 s1.mergeAndPrint(r1);
16 r1.mergeAndPrint(r2);
17 r2.mergeAndPrint(or1);
18 sr1.mergeAndPrint(or2);
19 sr2.mergeAndPrint(new Running(60.0));
```

**Domanda a)** Indicare le istruzioni che generano errore in compilazione o run-time, specificando il tipo di errore e la motivazione.

**Soluzione**

La riga 4 genera errore in compilazione perchè `Running` non è sottoclasse di `Swimming` e in compilatore non accetta il cast. In conseguenza dell'errore precedente non va considerata neppure la riga 14.

**Domanda b)** Avendo eliminato le righe con errore, scrivere cosa viene stampato per ogni riga rimanente.

**Soluzione**

12. Tot duration 120.0
13. Tot duration 130.0 (adding 10 extra for transition)
15. Tot duration 130.0 (adding 10 extra for transition)
16. Tot duration 120.0
17. Tot duration 120.0
18. Tot duration 125.0
19. Tot duration 120.0